

FineInstructions: Scaling Synthetic Instructions to Pre-Training Scale

A. Patel¹, C. Raffel^{2,3,4}, and C. Callison-Burch¹, 'FineInstructions: Scaling Synthetic Instructions to Pre-Training Scale', Jan. 29, 2026, arXiv: arXiv:2601.22146. doi: 10.48550/arXiv.2601.22146.

¹ Department of Computer Science and Information, University of Pennsylvania, Philadelphia, USA

² Department of Computer Science Toronto

³ Vector Institute, Toronto Canada

⁴ Hugging Face, Brooklyn USA

Abstract

- LLM's are trained on limited supervised data
 - This procedure is enhanced with smaller instruction-tuned data (examples of instruction and responses)
- Paper wants to overcome that problem by introducing the automatic generation of synthetic instruction and answer pairs
- Dataset: FineInstructions - ~18M instruction templates from real user data (queries and prompts)
- Instructions (Templates) is matched with human written documents
- This training procedure seems to outclass the traditional approach of self supervised word prediction learning
- [FineInstructions](#)

Introduction

Historical Derivation

- LLM's have traditionally been trained on "predict the next word" on large unstructured text data
 - Vast majority of time, computing and resources are spent on that task
- Supervised learning can enhance the capabilities with instruction - answer examples
 - This is called instruction tuning
 - Problem: Supervised instruction answer corpora are expensive and small
- Using Frontier Models (gpt etc.) used to generate instruction - answer examples
 - This approach only mimics the frontier model
- Conclusion:
 - In the end self supervised learning is the responsible for the models knowledge
 - Capabilities (answer a question, write a letter) are learned besides the factual knowledge
 - Question is: Is this the best way to train that kind of capabilities?
 - Alternative transformations pipelines (Manini et al. 2024; Su et al 2024) have been suggested

Approach in the paper

- Procedure called FineInstructions
 - Transformation of pre-training corpora into collections of instruction - response pairs
 - Generation of ~18M generic instruction templates derived from real user queries, which are subsequently instantiated on a per-document basis to create grounded Q&A pairs.
 - This procedure can be used for instruction tuning pre-model training
 - *Hypothesis*
 - This restructuring leads to better knowledge absorption and model performance
 - Simplify pre-training since low quality noise documents are handled before
 - Guarantees higher quality and more educational sections of a document
1. ~18M templates were created -> Represents highly diverse in-distribution synthetic data
 2. FineInstructions are introduced -> Converting pre-training documents into synthetic instruction - answer pairs
 3. Pre-training with FineInstructions outperforms standard pre-trainings

4. Release of a FineInstruction dataset of 1B+ synthetic instructions

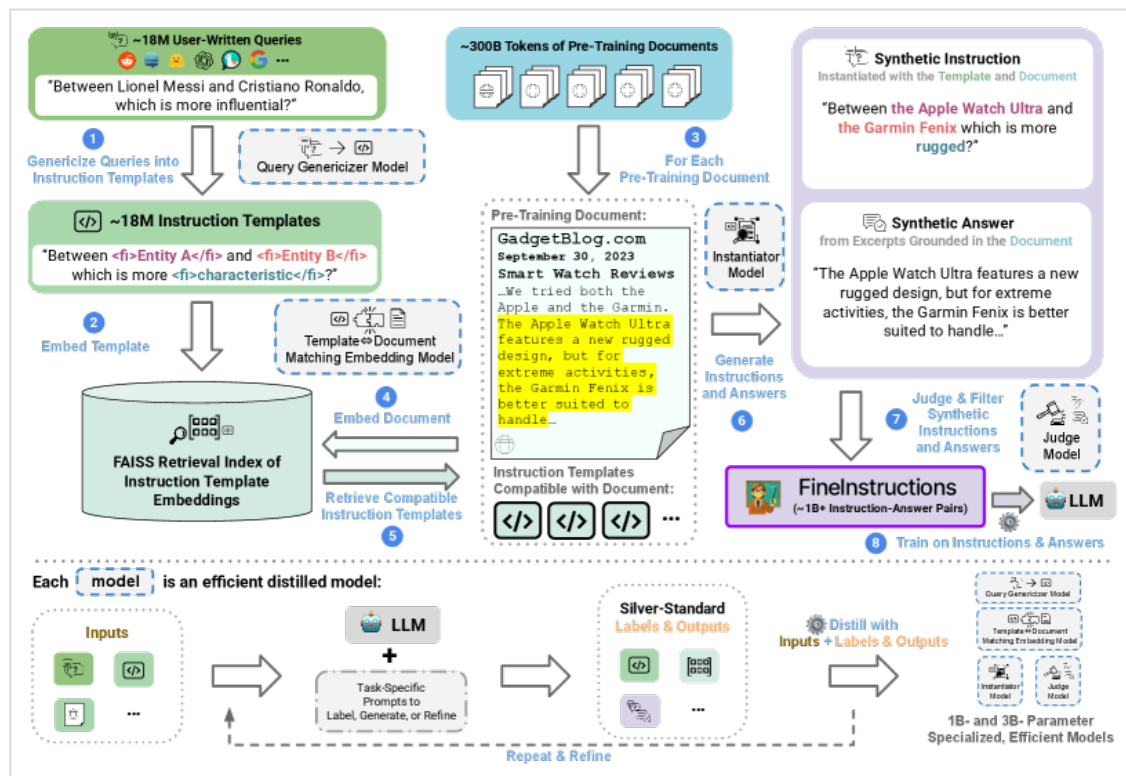


Figure 1. The FineInstructions pipeline for efficiently generating diverse, pre-training scale, synthetic instruction-answer pairs.

Related work

- Genesis
 - Training on unstructured text
 - Apply filter method
 - Re-sampling, re-wighting
 - Converting raw data into instruction-answer pairs
 - Rephrasing low-quality documents
 - Conversion raw data into instruction -answer pairs
 - Approaches with instruction templates were usually hand crafted thus lack in raw numbers
- This paper
 - Generation of synthetic data by automatically creating and instantiating instruction templates

| FinelInstructions

Overview

- Collection of user-written queries -> questions, instructions, task requests
- Generalizing them into generic instruction templates
- Match between pre-training document with instruction templates
 - Match if answer to query is provided and sufficient for template instantiation
- A judge model is used to measure the quality
- Pipeline Logic:
 - Implements a “weak supervision” approach; transforms unlabeled corpora into supervised datasets via programmatic labeling.
 - Silver-Standard: Llama-3.3 70B (Teacher) generates high-quality examples.
 - Distillation: Efficient models (Llama-3.2 1B/3B) are trained on silver-standard data to replicate teacher performance.
 - Scaling: Distilled models generate 1B+ instruction-answer pairs at pre-training scale.
 - Enable supervised pre-training from scratch.
 - Ensures the model is “in-distribution” with downstream usage (prompt-response) from the first token.

Generating Instruction Templates

- Collection of human written prompt templates from various libraries
 - Filter of harmful queries
 - Decontamination (Filter “known” questions)
- Convert queries to templates by inserting tags for entities or scenarios
 - Within tags a description of what might be appropriate is inserted
 - Later the spans are filled from a matched document
- ~50k queries were selected for the fine tuning of a Llama-3.2 1B Instruction model
 - templates were converted to “silver” synthetic pipelines by “prompting an LLM with a series of prompts
 - Description (what kind of document could be instantiated and answer) have been created
- With that ~18M templates were generated

Matching Documents to Instruction Templates

- Semantic model (BGE-M3) was used to embed all descriptions
- Corpus of unstructured documents were used to generate ~200k descriptions and embedded
- Store embeddings in FAISS (Fast semantic search db)
- Retrieve top 5 templates for a description
- Judge LLM labels for hard positives and hard negatives
- 2 step fine tuning of BGE-M3

Gaussian Pooling for Document Coverage

- Global mean pooling washes out specific information in long documents
- Solution: Generate K local embeddings (K=5) in addition to one global embedding
 - Centers are distributed linearly across the document length
 - Gaussian weights prioritize tokens near the center of each chunk
 - Training: BGE-M3 fine-tuned to reduce representation mixing and maintain local semantic clarity
 - Validation: High Pearson correlation (0.99) between retrieved chunk index and actual answer location in the document

Retrieval

- Match candidates selected via cosine similarity threshold (> 0.865)
- Weighted random sampling used to encourage template diversity
- Templates vary in complexity: simple (1-2 tags) vs. complex (10+ tags)
- Sampling weights matched to complexity distribution of real-world queries (WildChat, LMSys, OASst1, ShareLM)

Generating Instructions and Answers

- Groundedness (Fidelity over Hallucination)
 - Raw documents are converted into natural Q&A formats by allowing minor rephrasing or introductory words
 - Strict constraint: At least 80% of the generated answer must consist of direct excerpts from the source
- Compute Optimization (Tag-based Copying)
 - Token-by-token generation of long sequences is highly expensive and causes GPU memory bottlenecks

- Solution: The model generates short XML-like tags with ellipsis notation (e.g., start <...> end).
- These tags are programmatically expanded post-decoding, drastically reducing the number of generated tokens and computational overhead
- Two-Round Distillation Pipeline:
 - Round 1: Llama-3.3 70B generates ~100K silver-standard examples to fine-tune a Llama-3.2 3B Instruct model.
 - Round 2: The 3B model generates a massive pool of candidates, which are filtered by an LLM-as-judge to yield ~100K new, balanced, high-quality examples
 - Dataset Stratification: Active filtering (using keyword-based tools) ensures a balanced distribution of text length, complexity (variable count), and critical topics like math and code
 - Negative Training: In ~5% of cases, the model is trained to output “null” for incompatible pairs, preventing forced, low-quality hallucinations

Judging and Filtering Instructions and Answers

- Standalone Format Advantage
 - Structuring data as independent Q&A pairs makes it highly compatible with off-the-shelf reward and judge models
- Quality Control & Metrics
 - Model: Flow Judge (distilled 3.8B parameter judge model)
 - Evaluation: 5-point Likert scale (1 = irrelevant/off-topic, 5 = perfect response with no extraneous or repetitive content)
 - Filter Threshold: Retain only instruction-answer pairs scoring ≥ 4

Experimental Setup

Baselines

- Methods described are based on
 - same unstructured document source corpora
 - models are trained on the same number of tokens

- Baseline Categories:
 - Standard Pre-training: Models trained on the raw, unstructured “vanilla” source documents.
 - Synthetic & Rephrasing Methods: State-of-the-art procedures from prior work that transform raw pre-training documents.
 - Implementation: Reuses the exact vanilla corpora and pre-computed synthetic datasets released by the authors of the prior works to ensure a mathematically fair benchmark.

Instruction Pre-Training (IPT)

- Method - Synthesizer model converts documents into instruction-answer pairs
- Synthesizer Training - Trained on academic NLP Q&A datasets
- Corpus - ~23B tokens from RefinedWeb in both vanilla and transformed versions

Nemotron-CC

- Method - Processes filtered CommonCrawl documents using an LLM for multi-task generation
- Tasks - Document rephrasing, synthetic Q&A generation, and core knowledge extraction
- Corpus - ~300B tokens in both vanilla and synthetic mixture versions
- Direct Comparison Baselines
 - Pure Q&A - ~300B tokens of standalone Q&A data to compare directly with IPT and FineInstructions
 - WRAP Baseline - ~300B tokens of synthetically rephrased data using Web Rephrase Augmented Pre-training

Pre-Training

- Data Budgeting and Selection - Token Budget Constraints - Strict limit: Total token count of Q&A pairs cannot exceed the source document count - Rollover mechanism: Unused budget carries over to subsequent documents to ensure exact corpus alignment - Selection Pipeline - Pool generation: Pipeline creates 6 instruction-answer pairs per document - Accumulation logic: Pairs are randomly selected until the next addition would exceed the limit - Average Yield: Forced discard of ~50% leads to an average of ~3 retained pairs per document
 - Format and Structural Design
 - Template Structure: Chat template “Instruction: {{instruction}}: {{answer}}”
 - Standalone vs Contextual:
 - Baselines append Q&A to raw documents (reading-comprehension style)
 - FineInstructions uses standalone pairs (context-independent)

- Standalone Advantage: Eliminates redundant context and forces direct assistant alignment
- Training Configuration
 - Corpora Scales: ~23B tokens (IPT comparison) and ~300B tokens (Nemotron-CC comparison)
 - Hardware/Framework: Meta Lingua framework on 8xH100 GPUs
 - Model/Tokenizer: 1.8B parameter model using Llama-3 tokenizer
 - Epoch Allocation: 1 epoch for Nemotron-CC (300B) vs 4 epochs for IPT (23B) to ensure weight saturation

Benchmarks

- Evaluation Frameworks
 - MixEval
 - Composition: Subset of academic benchmarks (e.g., MMLU, TriviaQA) filtered for high correlation with human judgment
 - Mostly one correct answer
 - Evaluation: LLM-as-a-judge (GPT-5 mini) using both “Standard” and “Hard” splits
 - MT-Bench-101
 - Nature: Multi-turn benchmark for realistic, subjective user queries (opinions, advice)
 - Noisy output, Multiple prompts on vague topics (What’s the best for my career etc)
 - Evaluation: LLM-as-a-judge (GPT-5 mini) using a 10-point Likert rubric
 - AlpacaEval
 - Nature: Head-to-head comparison between two models based on realistic user tasks
 - More creative, subjective, categorizes based on helpfulness, style, formatting etc.
 - Evaluation: LLM-as-a-judge (GPT-4-Turbo) with built-in corrections for length-bias
- Evaluation Strategy
 - Prompting: Benchmark questions formatted into chat templates matching each model’s training template
 - Vanilla Baseline: Standard (Instruction:) and (Answer:) template used for models pre-trained on raw documents
 - Sampling: Greedy sampling utilized for all model response generations (Always use the top 1 word in prediction)

Results

- Performance Overview
 - FineInstructions consistently outperforms standard pre-training and prior synthetic baselines (IPT, Nemotron-CC) across all benchmarks
 - Significant performance gap: ~69% relative improvement on MixEval (vs IPT baseline) and ~39% on Nemotron-CC
- Qualitative Superiority
 - Generalization: FineInstructions shows better generalization on open-ended real-world tasks (AlpacaEval) compared to competitors focused on narrow academic formats
 - Academic Baselines: Prior methods (IPT) only outperform standard models if evaluation benchmarks align perfectly with their academic-style training data
- Model Scaling Efficiency (Appendix F)
 - Small Model Advantage: FineInstructions is highly effective for compute-efficient models (300M to 1.8B parameters)
 - Scaling Benchmark: A small model trained with FineInstructions reaches performance levels typically associated with significantly larger architectures

Table 1. Benchmark performance of 1.8B parameter models pre-trained on FineInstructions and various baselines. We **bold** the strongest method and **bold** the win rate % if FineInstructions wins.

Method	MixEval Acc (%)		MT-Bench-101 Likert Score	AlpacaEval FI Win Rate % (Δ Win Margin %)
	Standard	Hard		
<i>IPT Corpus (23B)</i>				
Standard Pre-Training	17.8	14.0	1.9	73.6% (Δ 47.2%)
IPT	19.8	16.7	2.4	68.2% (Δ 36.4%)
FineInstructions	31.7	19.2	2.8	–
<i>Nemotron-CC Corpus (300B)</i>				
Standard Pre-Training	24.0	17.1	3.5	63.6% (Δ 27.2%)
WRAP	22.8	18.4	3.6	65.1% (Δ 30.2%)
Q&A	27.1	18.9	3.4	76.1% (Δ 52.2%)
Nemotron-CC	24.5	16.7	3.6	65.9% (Δ 31.8%)
FineInstructions	33.0	21.8	3.9	–

Table 1. Benchmark performance of 1.8B parameter models pre-trained on FineInstructions and various baselines. We bold the strongest method and bold the win rate % if FineInstructions wins.

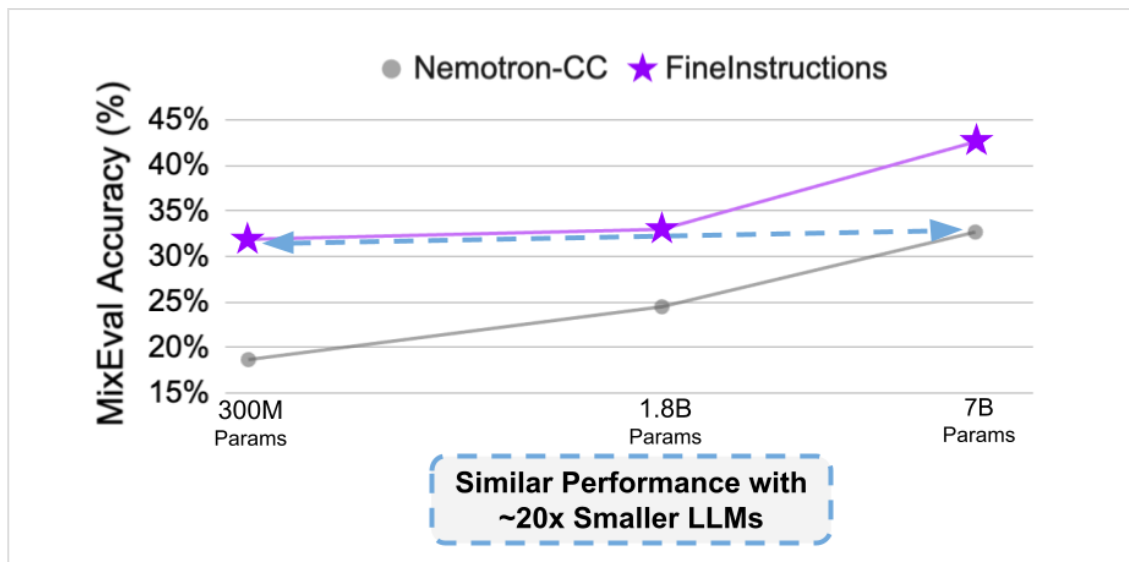


Figure 2. Efficiency from pre-training on FineInstructions data

Discussion

Diversity of Instructions

- Analysis reveals a power-law relationship (long-tail distribution where many unique templates appear infrequently, while a few simple ones dominate) between pre-training documents and unique instruction templates
- High template diversity: No single template accounts for >0.09% of total instructions
- Diverse task distribution: Templates cover various domains including science, code, and medicine

Impact of Judging and Filtering

- Filtering stage acts as a quality control mechanism, removing low-scoring Q&A pairs (threshold < 4/5)
- Confirmed performance boost on benchmarks, specifically AlpacaEval

Limitations and Future Directions

- Matching complexity: Retrieval of complex templates (10+ tags) remains a primary failure mode

- Computational bottleneck: Scaling the pipeline beyond the 3B parameter range is resource-intensive
- Benchmark limitations: Current LLM benchmarks focus on factual recall and are ill-suited for long-tail realistic knowledge tasks (advice, suggestions)
- Evaluation bias: High dependency on LLM-as-a-judge models introduces potential circularity and bias artifacts

Conclusion

- Core Contribution - Proposes FineInstructions as an effective pipeline for transforming real user queries into diverse, in-distribution synthetic data at scale
- Paradigm Shift - Demonstrates that pre-training on standalone instruction-answer pairs is superior to traditional self-supervised next-token prediction
- Knowledge Absorption - Restructuring data into Q&A formats improves the efficiency of knowledge absorption and better reflects real-world downstream usage
- Impact - Provides an optimized strategy for training capable, compute-efficient language models at smaller parameter scales

Models used in the paper

Model	Role / Designation	Primary Task	Purpose / Details
Llama-3.3 70B Instruct	Teacher Model	Silver-Standard Generation	Generates high-quality examples used for the distillation of smaller models.
Llama-3.2 1B Instruct	Query Genericizer	Template Creation	Transforms real-world user queries into generic instruction templates.
Llama-3.2 3B Instruct	Instantiator Model	Q&A Generation	Instantiates templates using source documents to produce grounded instruction-answer pairs.

Model	Role / Designation	Primary Task	Purpose / Details
BGE-M3	Embedding Model	Semantic Matching	Vectorizes documents and templates to enable efficient retrieval via FAISS.
Flow Judge (3.8B)	Final Judge	Quality Filtering	Evaluates generated pairs on a 5-point Likert scale; retains only high-quality samples (≥ 4).
Llama-3 Tokenizer	Tokenizer	Tokenization	Ensures consistent tokenization for the resulting pre-trained models.

Discussion and Open Questions

- Statistical Validity of Pearson Correlation
 - Authors report a 0.99 Pearson correlation between chunk index and answer location
 - Critique - Chunk index is a discrete ordinal variable (1 to 5) while position is continuous (0 to 100%)
 - Normal distribution assumption required for Pearson is likely violated due to U-shaped or non-linear information density in documents
 - Question - Would a non-parametric Spearman rank correlation be more mathematically robust to evaluate this relationship
- The Token Tax and Information Loss
 - Framing questions and formatting templates consumes 10-20% of the strict token budget
 - Question - Does this structural overhead cause critical knowledge loss in high-density documents like mathematics or code
 - Discussion - Is the trade-off of discarding web noise worth the quantitative loss of raw text tokens
- Limitations of Fixed Document Partitioning
 - Gaussian pooling utilizes a static $K=5$ chunk division regardless of document length

- Question - How does this static partition perform on extremely long documents such as textbooks where $K=5$ is highly insufficient
- Discussion - Should future pipelines implement a dynamic K -value based on token count or semantic density
- Lack of Clarity in Entity and Scenario Replacement
 - Section 3.1 states that queries are genericized by replacing spans with tags
 - Critique - The paper does not explain how these entity and scenario boundaries are detected or how descriptions are generated consistently
 - Discussion - The lack of explicit heuristics or token-classification rules makes this initial pipeline step a reproducibility black box
- Vague Prompting Methodology for Silver-Standard Data
 - Sourcing the initial 50K templates relies on “prompting an LLM with a series of prompts”
 - Critique - The exact prompting architecture (few-shot, chain-of-thought, or multi-step verification) is not defined in the main text
 - Discussion - Since the entire downstream genericizer model (1B) depends on this silver-standard dataset, the lack of prompt-engineering detail is a major documentation gap